Cheatsheet PE – MVC + Entity Framework (.NET 10)

Gebaseerd op **Oefening 15 - MVCVoertuig - Voorbeeld PE**. De volledig werkende oplossing staat in de map `MVCVoertuig/` van deze repo. Elke stap hieronder bevat de **volledige code** om te kopiëren.

⏱️ Tijdsplan: 60 minuten

Tijd	Doen	Stappen
0 – 3	Project aanmaken + packages (plak ze in de .csproj, zie stap 1)	1
3 – 15	Model, custom validatie, DbContext, appsettings, Program.cs (**kopiëren + namen vervangen**)	5, 6, 2, 3, 4
15 – 20	Seed data, `Add-Migration initial`, `Update-Database` → **F5: start de app?**	10, 9
20 – 30	Controller + Index/Create-views (Add View met template) → **F5 en test Create**	7, 8
30 – 35	Layout	11
35 – 40	Partial view + gebruiken in Index	12
40 – 47	TagHelper + `@addTagHelper`	13
47 – 55	ViewComponent + Default.cshtml	14
55 – 60	Testen, opslaan, indienen	

Regels tijdens de PE

1. **Kopieer volledige bestanden** en vervang daarna de namen met Ctrl+H. Overtypen kost te veel tijd.
2. **Zit je meer dan 5 minuten vast? Ga door naar de volgende stap.** Je komt er op het einde op terug. Stap 11–14 hangen bijna niet af van elkaar.
3. Druk na elk blok op **Ctrl+Shift+B** (bouwen). Fouten die je meteen ziet, los je in 1 minuut op. Na 20 minuten duurt dat veel langer.
4. Bouwt iets niet en heb je geen tijd meer? **Zet die code in commentaar** (selecteer → Ctrl+K, Ctrl+C), zodat de rest van het project wel bouwt en draait.
5. Vanaf minuut 55 schrijf je geen nieuwe code meer. Test dan en dien in.

Vanavond voorbereiden (belangrijk!)

- [ ] Download deze PDF **lokaal**, zodat je er zonder internet aan kunt.
- [ ] Bouw de oefening één keer helemaal met alleen deze cheatsheet en **meet je tijd**.
- [ ] Controleer of LocalDB werkt op je laptop: doe `Update-Database` en F5 in een project.
- [ ] Vraag (of check) of je tijdens de PE **je eigen projecten** mag openen. Mag dat, open dan `MVCVoertuig` en kopieer hele bestanden: dat is het snelst.
- [ ] Is er tijdens de PE geen internet? Dan werkt NuGet *Browse* niet, maar het plakken van de packages in de `.csproj` (stap 1) wel. Ze komen dan uit de cache, op voorwaarde dat je die versie al eens geïnstalleerd hebt (bv. vanavond).

0. Eerst: namen vervangen

De PE heeft waarschijnlijk dezelfde stappen, maar met een ander onderwerp. Kopieer de code en doe in Visual Studio **Ctrl+H** (Replace, *Match case* AAN). Vervang daarbij **eerst het meervoud, dan het enkelvoud**.

In deze cheatsheet	Wordt in de PE (voorbeeld)
`MVCVoertuig` (projectnaam/namespace)	`MVCBoek`
`Voertuigen` (tabel / DbSet)	`Boeken`
`Voertuig` (model, controller)	`Boek`
`VoertuigId`	`BoekId`
`VoertuigDbContext`	`BoekDbContext`
`VoertuigConnection` / `VoertuigDb`	`BoekConnection` / `BoekDb`
`Merk` (het veld voor de ViewComponent)	bv. `Auteur`, `Genre`, `Type`

⚠️ De **namespace** moet overal gelijk zijn aan je projectnaam. Kies bij *New Project* dus exact de naam uit de opgave.

Nieuw project: File → New → Project → *ASP.NET Core Web App (Model-View-Controller)* → naam → Framework **.NET 10** → Authentication **None** → Create.

---

1. NuGet-packages

Rechtsklik op het project → **Manage NuGet Packages** → tab *Browse* → installeer (versie 10.x):

- `Microsoft.EntityFrameworkCore.SqlServer`
- `Microsoft.EntityFrameworkCore.Tools`

Klopt het? Dubbelklik op het project; in de `.csproj` staan dan 2 `<PackageReference>`-regels.

⚡ **Sneller (en werkt ook offline):** dubbelklik op het project (de `.csproj` opent) en plak dit **binnen** `<Project>`, onder `</PropertyGroup>`. Sla op; Visual Studio herstelt de packages dan zelf.

```
  <ItemGroup>
    <PackageReference Include="Microsoft.EntityFrameworkCore.SqlServer" Version="10.0.14" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.Tools" Version="10.0.14">
      <PrivateAssets>all</PrivateAssets>
      <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive</IncludeAssets>
    </PackageReference>
  </ItemGroup>
```

Stopt `Add-Migration` met een waarschuwing over kwetsbare packages (NU1903)? Zet dan in de `<PropertyGroup>` de regel `<NuGetAuditMode>direct</NuGetAuditMode>`. Dat is hetzelfde als in je MVCSportStore-project.

---

2. DbContext

Rechtsklik op het project → Add → New Folder `Data` → Add → Class `VoertuigDbContext.cs`

```
using Microsoft.EntityFrameworkCore;
using MVCVoertuig.Models;

namespace MVCVoertuig.Data
{
    public class VoertuigDbContext : DbContext
    {
        public VoertuigDbContext(DbContextOptions<VoertuigDbContext> options) : base(options) { }

        public DbSet<Voertuig> Voertuigen { get; set; } = null!;   // tabelnaam = "Voertuigen"
    }
}
```

(Rode lijn onder `Voertuig`? Dat is normaal tot stap 5 klaar is.)

---

3. appsettings.json

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "ConnectionStrings": {
    "VoertuigConnection": "Server=
(localdb)\\MSSQLLocalDB;Database=VoertuigDb;Trusted_Connection=True;MultipleActiveResultSets=true"
  }
}
```

⚠️ Schrijf **2 backslashes** `\\` en zet een **komma** na `"AllowedHosts": "*"`.

---

4. Program.cs (volledig, inclusief stap 10)

```
using Microsoft.EntityFrameworkCore;
using MVCVoertuig.Data;

var builder = WebApplication.CreateBuilder(args);

// Stap 4: DbContext opstarten
var connectionString = builder.Configuration.GetConnectionString("VoertuigConnection");
builder.Services.AddDbContext<VoertuigDbContext>(
    options => options.UseSqlServer(connectionString));

builder.Services.AddControllersWithViews();

var app = builder.Build();

if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseRouting();
app.UseAuthorization();
app.MapStaticAssets();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}")
    .WithStaticAssets();

// Stap 10: default data (na Build, vóór Run)
SeedData.EnsurePopulated(app);

app.Run();
```

Moet de app meteen op de voertuigenpagina starten? Gebruik dan `pattern: "{controller=Voertuig}/{action=Index}/{id?}"`.

---

5. Model

`Models/Voertuig.cs`

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
using MVCVoertuig.Validation;

namespace MVCVoertuig.Models
{
    public class Voertuig
    {
        public int VoertuigId { get; set; }                    // Primary key (KlasseNaam + Id)

        [Required(ErrorMessage = "Merk is verplicht")]
        [StringLength(100)]
        public string Merk { get; set; } = string.Empty;

        [Required(ErrorMessage = "MerkType is verplicht")]
        [StringLength(100)]
        public string MerkType { get; set; } = string.Empty;

        [Required(ErrorMessage = "Bouwjaar is verplicht")]
        [Bouwjaar]                                             // custom validatie (stap 6)
        public int Bouwjaar { get; set; }

        [Required(ErrorMessage = "Km is verplicht")]
        [Range(0, 5000000, ErrorMessage = "Km moet tussen 0 en 5000000 liggen")]
        public int Km { get; set; }

        [Required(ErrorMessage = "VerkoopPrijs is verplicht")]
        [Column(TypeName = "decimal(8,2)")]
        public decimal VerkoopPrijs { get; set; }

        public bool AanwezigInShowroom { get; set; }           // niet verplicht
    }
}
```

Vertaaltabel: opgave → annotatie

In de opgave	Code
Primary key	`public int XId { get; set; }` (of `[Key]` boven een andere naam)
Verplicht	`[Required]`
Niet verplicht (string)	`public string? Naam { get; set; }` (met `?`)
Niet verplicht (int/decimal/DateTime)	`public int? Aantal { get; set; }` (met `?`)
Niet verplicht (bool)	`public bool X { get; set; }` (geen attribuut)
100 karakters	`[StringLength(100)]` of `[MaxLength(100)]`
Min 2, max 50 karakters	`[StringLength(50, MinimumLength = 2)]`
Waarde tussen a en b	`[Range(0, 5000000)]`
Decimal (8,2)	`[Column(TypeName = "decimal(8,2)")]`
Datum (alleen datum)	`[DataType(DataType.Date)] public DateTime Datum { get; set; }`
E-mail	`[EmailAddress]`
Telefoon / URL	`[Phone]` / `[Url]`
Patroon (bv. 4 cijfers)	`[RegularExpression(@"^\d{4}$", ErrorMessage = "...")]`
Andere labeltekst	`[Display(Name = "Verkoopprijs")]`
Niet in de databank	`[NotMapped]`
Andere tabelnaam	`[Table("Wagens")]` boven de klasse

---

6. Custom model validatie

Maak de map `Validation` aan met `BouwjaarAttribute.cs`.

```
using System.ComponentModel.DataAnnotations;

namespace MVCVoertuig.Validation
{
    public class BouwjaarAttribute : ValidationAttribute
    {
        protected override ValidationResult? IsValid(object? value, ValidationContext validationContext)
        {
            int bouwjaar = Convert.ToInt32(value);
            int huidigJaar = DateTime.Now.Year;

            if (bouwjaar < 1900 || bouwjaar > huidigJaar)
            {
                return new ValidationResult($"Bouwjaar moet tussen 1900 en {huidigJaar} liggen");
            }
            return ValidationResult.Success;
        }
    }
}
```

In het model gebruik je `[Bouwjaar]`. De naam zonder "Attribute" volstaat.

Varianten die je kunt krijgen

Met parameters, gebruik: `[JaarTussen(1900)]`

```
public class JaarTussenAttribute : ValidationAttribute
{
    private readonly int _minJaar;
    public JaarTussenAttribute(int minJaar) { _minJaar = minJaar; }

    protected override ValidationResult? IsValid(object? value, ValidationContext validationContext)
    {
        int jaar = Convert.ToInt32(value);
        if (jaar < _minJaar || jaar > DateTime.Now.Year)
            return new ValidationResult($"Jaar moet tussen {_minJaar} en {DateTime.Now.Year} liggen");
        return ValidationResult.Success;
    }
}
```

Datum niet in de toekomst, gebruik: `[NietInToekomst]`

```
public class NietInToekomstAttribute : ValidationAttribute
{
    protected override ValidationResult? IsValid(object? value, ValidationContext validationContext)
    {
        if (value is DateTime datum && datum > DateTime.Now)
            return new ValidationResult("Datum mag niet in de toekomst liggen");
        return ValidationResult.Success;
    }
}
```

Minimumleeftijd (bv. 18 jaar)

```
if (value is DateTime geboorte && geboorte.AddYears(18) > DateTime.Today)
    return new ValidationResult("Je moet minstens 18 jaar zijn");
```

Tekst moet met een hoofdletter beginnen

```
var tekst = value as string;
if (string.IsNullOrEmpty(tekst) || !char.IsUpper(tekst[0]))
    return new ValidationResult("Moet met een hoofdletter beginnen");
```

Twee velden samen controleren: model `: IValidatableObject`

```
public class Voertuig : IValidatableObject
{
    // ... properties ...
    public IEnumerable<ValidationResult> Validate(ValidationContext validationContext)
    {
        if (Km > 0 && Bouwjaar == DateTime.Now.Year && Km > 50000)
            yield return new ValidationResult("Te veel km voor een nieuw voertuig", new[] { nameof(Km) });
    }
}
```

Fout toevoegen in de controller:

```
if (_context.Voertuigen.Any(v => v.MerkType == voertuig.MerkType))
    ModelState.AddModelError("MerkType", "Dit type bestaat al");
```

Een custom validatie werkt alleen **server-side**. De fout verschijnt pas na het klikken op Opslaan, en dat is normaal.

---

7. Lege controller met Index

Rechtsklik op `Controllers` → Add → Controller → **MVC Controller – Empty** → `VoertuigController.cs` (De klasse moet eindigen op `Controller`. In URL's en `asp-controller` gebruik je alleen `Voertuig`.)

8. Controller met Index en Create (volledig)

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using MVCVoertuig.Data;
using MVCVoertuig.Models;

namespace MVCVoertuig.Controllers
{
    public class VoertuigController : Controller
    {
        private readonly VoertuigDbContext _context;

        public VoertuigController(VoertuigDbContext context)
        {
            _context = context;
        }

        // GET /Voertuig   of   /Voertuig?merk=BMW
        public async Task<IActionResult> Index(string? merk)
        {
            var voertuigen = _context.Voertuigen.AsQueryable();
            if (!string.IsNullOrEmpty(merk))
            {
                voertuigen = voertuigen.Where(v => v.Merk == merk);
            }
            ViewBag.Merk = merk;
            return View(await voertuigen.ToListAsync());
        }

        [HttpGet]
        public IActionResult Create()
        {
            return View();
        }

        [HttpPost]
        [ValidateAntiForgeryToken]
        public async Task<IActionResult> Create(Voertuig voertuig)
        {
            if (ModelState.IsValid)
            {
                _context.Voertuigen.Add(voertuig);
                await _context.SaveChangesAsync();
                return RedirectToAction(nameof(Index));
            }
            return View(voertuig);
        }
    }
}
```

Views genereren (snelste manier)

Rechtsklik in de methode `Index()` → **Add View** → *Razor View* →

- View name `Index`, Template **List**, Model class **Voertuig** → Add
- Doe hetzelfde voor `Create()` met Template **Create**.

Visual Studio maakt dan de tabel en het formulier met validatie.

Liever zonder template? Gebruik dan deze `Views/Voertuig/Create.cshtml`:

```
@model Voertuig
@{ ViewData["Title"] = "Nieuw voertuig"; }

<h1>Nieuw voertuig</h1>
<form asp-action="Create">
    <div asp-validation-summary="ModelOnly" class="text-danger"></div>

    <div class="mb-2">
        <label asp-for="Merk" class="control-label"></label>
        <input asp-for="Merk" class="form-control" />
        <span asp-validation-for="Merk" class="text-danger"></span>
    </div>
    @* ... zelfde blok voor MerkType, Bouwjaar, Km, VerkoopPrijs ... *@

    <div class="form-check mb-3">
        <input asp-for="AanwezigInShowroom" class="form-check-input" />
        <label asp-for="AanwezigInShowroom" class="form-check-label"></label>
    </div>

    <input type="submit" value="Opslaan" class="btn btn-primary" />
</form>
<a asp-action="Index">Terug</a>

@section Scripts {
    @{ await Html.RenderPartialAsync("_ValidationScriptsPartial"); }
}
```

Moet je Edit, Details of Delete maken? Gebruik dan **Add → Controller → MVC Controller with views, using Entity Framework**, met Model `Voertuig` en DbContext `VoertuigDbContext`. Dat genereert alle CRUD-code en views. (Bestaat de controller al? Kies dan een andere naam of verwijder de oude eerst.)

---

9. Tabel aanmaken (migrations)

Ga naar **Tools → NuGet Package Manager → Package Manager Console**:

```
Add-Migration initial
Update-Database
```

Controle: View → **SQL Server Object Explorer** → (localdb)\MSSQLLocalDB → Databases → VoertuigDb → Tables → dbo.Voertuigen

Commando	Doet
`Add-Migration Naam`	nieuwe migration na een wijziging aan een model
`Update-Database`	migrations uitvoeren op de databank
`Remove-Migration`	laatste (nog niet toegepaste) migration weg
`Drop-Database`	databank verwijderen (daarna opnieuw `Update-Database`)

Pas je het model later aan? Doe dan **opnieuw** `Add-Migration Wijziging` + `Update-Database`.

---

10. Default data

`Data/SeedData.cs`

```
using MVCVoertuig.Models;

namespace MVCVoertuig.Data
{
    public static class SeedData
    {
        public static void EnsurePopulated(WebApplication app)
        {
            using (var scope = app.Services.CreateScope())
            {
                var context = scope.ServiceProvider.GetRequiredService<VoertuigDbContext>();

                if (!context.Voertuigen.Any())
                {
                    context.Voertuigen.AddRange(
                        new Voertuig { Merk = "AUDI", MerkType = "A4", Bouwjaar = 2019, Km = 85000, VerkoopPrijs =
24950m, AanwezigInShowroom = true },
                        new Voertuig { Merk = "AUDI", MerkType = "Q5", Bouwjaar = 2021, Km = 42000, VerkoopPrijs =
39900m },
                        new Voertuig { Merk = "BMW", MerkType = "320d", Bouwjaar = 2018, Km = 120000, VerkoopPrijs =
18500m, AanwezigInShowroom = true },
                        new Voertuig { Merk = "BMW", MerkType = "X3", Bouwjaar = 2022, Km = 30000, VerkoopPrijs =
45750m },
                        new Voertuig { Merk = "PORSCHE", MerkType = "911", Bouwjaar = 2020, Km = 25000, VerkoopPrijs =
119000m });
                    context.SaveChanges();
                }
            }
        }
    }
}
```

In Program.cs zet je: `SeedData.EnsurePopulated(app);` (al opgenomen in stap 4). Geef **geen** Id op, want die maakt de databank zelf aan. Decimals schrijf je met `m` (`24950m`).

Alternatief met HasData in de DbContext (dan Add-Migration opnieuw; hier zijn de Id's **wel** verplicht):

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    base.OnModelCreating(modelBuilder);
    modelBuilder.Entity<Voertuig>().HasData(
        new Voertuig { VoertuigId = 1, Merk = "AUDI", MerkType = "A4", Bouwjaar = 2019, Km = 85000, VerkoopPrijs =
24950m });
}
```

11. Styling: `Views/Shared/_Layout.cshtml` (volledig)

```
<!DOCTYPE html>
<html lang="nl">
<head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>@ViewData["Title"] - MVCVoertuig</title>
    <link rel="stylesheet" href="~/lib/bootstrap/dist/css/bootstrap.min.css" />
    <link rel="stylesheet" href="~/css/site.css" asp-append-version="true" />
</head>
<body>
    <header>
        <nav class="navbar navbar-expand-sm bg-primary navbar-light">
            <a class="btn btn-info m-1" asp-controller="Home" asp-action="Index">Home</a>
            <a class="btn btn-info m-1" asp-controller="Voertuig" asp-action="Index">Voertuigen</a>
            @await Component.InvokeAsync("HeaderMenu")
        </nav>
    </header>
    <div class="container">
        <main role="main" class="pb-3">
            @RenderBody()
        </main>
    </div>
    <footer class="border-top footer text-muted">
        <div class="container">
            &copy; PXL - MVCVoertuig
        </div>
    </footer>
    <script src="~/lib/jquery/dist/jquery.min.js"></script>
    <script src="~/lib/bootstrap/dist/js/bootstrap.bundle.min.js"></script>
    <script src="~/js/site.js" asp-append-version="true"></script>
    @await RenderSectionAsync("Scripts", required: false)
</body>
</html>
```

De opgave geeft voor de link `class="link-warning"`. Kopieer exact wat in de opgave staat. Op de screenshots zijn het knoppen (`btn btn-info m-1`). ⚠️ Verwijder de `<script>`-regels en `RenderSectionAsync` **niet**. Zonder die regels werken Bootstrap en de validatie niet meer.

---

12. Partial View

Rechtsklik op `Views/Shared` → Add → View → Razor View – Empty → naam `_VoertuigCard.cshtml`

```
@model Voertuig

<div class="card card-outline-primary border-primary m-1 p-1">
    <div class="bg-faded p-1">
        <h4>
            @Model.Merk
            <span class="badge bg-primary" style="float:right">
                @Model.MerkType
            </span>
        </h4>
    </div>
    <div class="card-text p-1">@Model.Bouwjaar - @Model.Km</div>
</div>
```

`Views/Voertuig/Index.cshtml` (vervangt de List-tabel):

```
@model IEnumerable<Voertuig>
@{ ViewData["Title"] = "Voertuigen"; }

<h1>Voertuigen @ViewBag.Merk</h1>
<a class="btn btn-primary" asp-action="Create">Nieuw voertuig</a>

<car-info aantal="@Model.Count()"></car-info>

@foreach (var voertuig in Model)
{
    <partial name="_VoertuigCard" model="voertuig" />
}
```

Andere schrijfwijze: `@await Html.PartialAsync("_VoertuigCard", voertuig)`

---

13. TagHelper

Maak de map `TagHelpers` aan met `CarInfoTagHelper.cs`.

```
using Microsoft.AspNetCore.Razor.TagHelpers;

namespace MVCVoertuig.TagHelpers
{
    // <car-info aantal="5"></car-info>
    public class CarInfoTagHelper : TagHelper
    {
        public int Aantal { get; set; }
        public string Tekst { get; set; } = "Aantal voertuigen";

        public override void Process(TagHelperContext context, TagHelperOutput output)
        {
            output.TagName = "div";
            output.TagMode = TagMode.StartTagAndEndTag;
            output.Attributes.SetAttribute("class", "btn btn-outline-primary m-3");
            output.Content.SetHtmlContent($"{Tekst} <span class=\"badge bg-primary\">{Aantal}</span>");
        }
    }
}
```

⚠️ **VERPLICHT** in `Views/_ViewImports.cshtml`:

```
@using MVCVoertuig
@using MVCVoertuig.Models
@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers
@addTagHelper *, MVCVoertuig
```

Gebruik, bv. in `Views/Home/Index.cshtml`:

```
<div class="text-center">
    <car-info aantal="@ViewBag.AantalVoertuigen"></car-info>
    <br />
    <car-info aantal="@ViewBag.AantalInShowroom" tekst="Aantal voertuigen in showroom"></car-info>
</div>
```

Met in `HomeController`:

```
private readonly VoertuigDbContext _context;
public HomeController(VoertuigDbContext context) { _context = context; }

public IActionResult Index()
{
    ViewBag.AantalVoertuigen = _context.Voertuigen.Count();
    ViewBag.AantalInShowroom = _context.Voertuigen.Count(v => v.AanwezigInShowroom);
    return View();
}
```

(Vergeet `using MVCVoertuig.Data;` niet.)

Naamregels

C#	HTML
klasse `CarInfoTagHelper`	`<car-info>`
property `Aantal`	`aantal="..."`
property `BadgeKleur`	`badge-kleur="..."`
`[HtmlTargetElement("auto-info")]` boven de klasse	eigen tagnaam `<auto-info>`
`[HtmlTargetElement("a", Attributes = "kleur")]`	werkt op bestaande `<a kleur="danger">`

Varianten

Een volledig object doorgeven: `<voertuig-info voertuig="@item"></voertuig-info>`

```
public class VoertuigInfoTagHelper : TagHelper
{
    public Voertuig Voertuig { get; set; } = new();
    public override void Process(TagHelperContext context, TagHelperOutput output)
    {
        output.TagName = "span";
        output.Attributes.SetAttribute("class", "badge bg-success");
        output.Content.SetContent($"{Voertuig.Merk} {Voertuig.MerkType} ({Voertuig.Bouwjaar})");
    }
}
```

(Daarvoor heb je `using MVCVoertuig.Models;` nodig.)

Met TagBuilder:

```
using Microsoft.AspNetCore.Mvc.Rendering;
// ...
TagBuilder badge = new TagBuilder("span");
badge.AddCssClass("badge bg-primary");
badge.InnerHtml.Append(Aantal.ToString());

output.TagName = "div";
output.Attributes.SetAttribute("class", "btn btn-outline-primary");
output.Content.SetContent("Aantal voertuigen ");
output.Content.AppendHtml(badge);
```

Kleur afhankelijk van een waarde:

```
string kleur = Aantal > 3 ? "bg-success" : "bg-danger";
```

Op een bestaande `<a>`-tag:

```
[HtmlTargetElement("a", Attributes = "kleur")]
public class KnopTagHelper : TagHelper
{
    public string Kleur { get; set; } = "primary";
    public override void Process(TagHelperContext context, TagHelperOutput output)
        => output.Attributes.SetAttribute("class", $"btn btn-{Kleur} m-1");
}
// <a kleur="warning" asp-controller="Voertuig" asp-action="Index">Voertuigen</a>
```

- `SetContent` escapet HTML. Gebruik `SetHtmlContent` als je zelf tags (`<span>`) meegeeft.
- `output.PreContent.SetHtmlContent("...")` zet inhoud **vóór** de bestaande inhoud, `output.PostContent` **erna**.

---

14. ViewComponent

Maak de map `ViewComponents` aan met `HeaderMenuViewComponent.cs`.

```
using Microsoft.AspNetCore.Mvc;
using MVCVoertuig.Data;

namespace MVCVoertuig.ViewComponents
{
    public class HeaderMenuViewComponent : ViewComponent
    {
        private readonly VoertuigDbContext _context;

        public HeaderMenuViewComponent(VoertuigDbContext context)
        {
            _context = context;
        }

        public IViewComponentResult Invoke()
        {
            var merken = _context.Voertuigen
                .Select(v => v.Merk)
                .Distinct()            // alleen unieke merken
                .OrderBy(m => m)
                .ToList();
            return View(merken);
        }
    }
}
```

De view komt **exact** op dit pad (maak de mappen zelf aan): `Views/Shared/Components/HeaderMenu/Default.cshtml`

```
@model IEnumerable<string>

@foreach (var merk in Model)
{
    <a class="btn btn-info m-1" asp-controller="Voertuig" asp-action="Index" asp-route-merk="@merk">@merk</a>
}
```

Aanroepen in `_Layout.cshtml` (in de `<nav>`), met een van deze twee:

```
@await Component.InvokeAsync("HeaderMenu")
<vc:header-menu></vc:header-menu>
```

`asp-route-merk="AUDI"` maakt de link `/Voertuig?merk=AUDI`. Dat komt binnen als parameter `string? merk` in `Index` (zie stap 8).

Varianten

Met een parameter, bv. de voertuigen van één merk tonen:

```
public class VoertuigLijstViewComponent : ViewComponent
{
    private readonly VoertuigDbContext _context;
    public VoertuigLijstViewComponent(VoertuigDbContext context) { _context = context; }

    public async Task<IViewComponentResult> InvokeAsync(string merk)
    {
        var lijst = await _context.Voertuigen.Where(v => v.Merk == merk).ToListAsync();
        return View(lijst);   // Views/Shared/Components/VoertuigLijst/Default.cshtml  (@model IEnumerable<Voertuig>)
    }
}
```

```
@await Component.InvokeAsync("VoertuigLijst", new { merk = "BMW" })
<vc:voertuig-lijst merk="BMW"></vc:voertuig-lijst>
```

(`ToListAsync` heeft `using Microsoft.EntityFrameworkCore;` nodig.)

Merk + aantal per merk (bv. "AUDI (2)"):

```
var merken = _context.Voertuigen
    .GroupBy(v => v.Merk)
    .Select(g => new MerkAantal { Merk = g.Key, Aantal = g.Count() })
    .OrderBy(m => m.Merk)
    .ToList();
return View(merken);
```

```
// ViewModels/MerkAantal.cs
public class MerkAantal { public string Merk { get; set; } = ""; public int Aantal { get; set; } }
```

```
@model IEnumerable<MVCVoertuig.ViewModels.MerkAantal>
@foreach (var m in Model)
{
    <a class="btn btn-info m-1" asp-controller="Voertuig" asp-action="Index" asp-route-merk="@m.Merk">
        @m.Merk <span class="badge bg-primary">@m.Aantal</span>
    </a>
}
```

Het actieve merk markeren:

```
ViewBag.Actief = Request.Query["merk"].ToString();
```

```
<a class="btn @(merk == ViewBag.Actief ? "btn-warning" : "btn-info") m-1" ...>@merk</a>
```

---

15. LINQ queries (week 7)

Zet deze code in een controller-actie (bv. `Index`), of in een ViewComponent.

```
_context.Voertuigen.ToList();                                    // alles
_context.Voertuigen.Find(id);                                    // op primary key
_context.Voertuigen.FirstOrDefault(v => v.VoertuigId == id);     // eerste of null
_context.Voertuigen.Where(v => v.Merk == "BMW").ToList();        // filteren
_context.Voertuigen.Where(v => v.AanwezigInShowroom && v.Km < 100000).ToList();
_context.Voertuigen.Where(v => v.Merk.Contains(zoek)).ToList();  // zoeken
_context.Voertuigen.OrderBy(v => v.Merk).ThenBy(v => v.MerkType).ToList();
_context.Voertuigen.OrderByDescending(v => v.VerkoopPrijs).ToList();
_context.Voertuigen.OrderByDescending(v => v.VerkoopPrijs).Take(3).ToList(); // top 3 duurste
_context.Voertuigen.Select(v => v.Merk).Distinct().ToList();     // unieke waarden
_context.Voertuigen.Count();                                     // aantal
_context.Voertuigen.Count(v => v.AanwezigInShowroom);            // aantal met voorwaarde
_context.Voertuigen.Sum(v => v.VerkoopPrijs);                    // som
_context.Voertuigen.Average(v => v.VerkoopPrijs);                // gemiddelde
_context.Voertuigen.Max(v => v.Km);  _context.Voertuigen.Min(v => v.Bouwjaar);
_context.Voertuigen.Any(v => v.Km > 200000);                     // bestaat er één? (true/false)
_context.Voertuigen.Skip((pagina - 1) * 3).Take(3).ToList();     // paging, 3 per pagina
```

Groeperen:

```
var perMerk = _context.Voertuigen
    .GroupBy(v => v.Merk)
    .Select(g => new MerkStatistiek
    {
        Merk = g.Key,
        Aantal = g.Count(),
        GemiddeldePrijs = g.Average(v => v.VerkoopPrijs)
    })
    .ToList();
return View(perMerk);   // @model IEnumerable<MerkStatistiek>
```

⚠️ Wil je het resultaat naar een view sturen, gebruik dan een **ViewModel-klasse** (`new MerkStatistiek {...}`) en **geen** `new { ... }`.

Query-syntax (geeft hetzelfde resultaat):

```
var bmws = (from v in _context.Voertuigen
            where v.Merk == "BMW"
            orderby v.VerkoopPrijs descending
            select v).ToList();
```

Zoekformulier in Index.cshtml:

```
<form asp-action="Index" method="get" class="d-flex mb-2">
    <input name="zoek" value="@ViewBag.Zoek" class="form-control me-2" placeholder="Zoek merk..." />
    <button type="submit" class="btn btn-primary">Zoek</button>
</form>
```

```
public IActionResult Index(string? zoek)
{
    var lijst = _context.Voertuigen.AsQueryable();
    if (!string.IsNullOrEmpty(zoek)) lijst = lijst.Where(v => v.Merk.Contains(zoek));
    ViewBag.Zoek = zoek;
    return View(lijst.ToList());
}
```

16. Extra: data naar de view sturen

Manier	Controller	View
Model	`return View(lijst);`	`@model IEnumerable<Voertuig>` en `@Model`
ViewBag	`ViewBag.Titel = "x";`	`@ViewBag.Titel`
ViewData	`ViewData["Titel"] = "x";`	`@ViewData["Titel"]`
TempData (overleeft een redirect)	`TempData["Melding"] = "Opgeslagen";`	`@TempData["Melding"]`

Return-types: `View()`, `View(model)`, `RedirectToAction("Index")`, `RedirectToAction("Index", "Home")`, `NotFound()`, `Content("tekst")`, `Json(obj)`, `PartialView("_X", model)`.

Dropdown (bv. bij een foreign key, of een vaste lijst merken):

```
ViewBag.Merken = new SelectList(new[] { "AUDI", "BMW", "PORSCHE" });
// of uit de DB:  ViewBag.GarageId = new SelectList(_context.Garages, "GarageId", "Naam");
```

```
<select asp-for="Merk" asp-items="ViewBag.Merken" class="form-select"></select>
```

Zet de SelectList **ook** in de POST-actie voor `return View(voertuig);`, anders crasht de pagina bij een validatiefout.

Geld en getallen tonen: `@Model.VerkoopPrijs.ToString("C")` (€), `@Model.Km.ToString("N0")` (85.000)

Display-helpers (List-template): `@Html.DisplayNameFor(model => model.Merk)` en `@Html.DisplayFor(modelItem => item.Merk)`

17. Extra: relatie 1-n (als de PE 2 tabellen heeft)

```
public class Garage
{
    public int GarageId { get; set; }
    [Required, StringLength(100)] public string Naam { get; set; } = string.Empty;
    public List<Voertuig> Voertuigen { get; set; } = new();     // 1 garage -> n voertuigen
}
// In Voertuig:
public int GarageId { get; set; }          // foreign key
public Garage? Garage { get; set; }        // navigation property
```

In de DbContext: `public DbSet<Garage> Garages { get; set; } = null!;`. Daarna `Add-Migration` + `Update-Database`.

```
_context.Voertuigen.Include(v => v.Garage).ToList();      // garage mee ophalen
@item.Garage?.Naam                                         // in de view
```

Fluent API (alternatief voor attributen, in de DbContext):

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    base.OnModelCreating(modelBuilder);
    modelBuilder.Entity<Voertuig>(e =>
    {
        e.Property(v => v.Merk).IsRequired().HasMaxLength(100);
        e.Property(v => v.VerkoopPrijs).HasColumnType("decimal(8,2)");
        e.HasOne(v => v.Garage).WithMany(g => g.Voertuigen).HasForeignKey(v => v.GarageId);
    });
}
```

18. Foutmeldingen → oplossing

Probleem	Oplossing
`Add-Migration` is niet herkend	Package `Microsoft.EntityFrameworkCore.Tools` installeren en VS herstarten
`Unable to create a 'DbContext'` bij Add-Migration	`AddDbContext` ontbreekt in Program.cs, de naam van de connection string klopt niet, of het project bouwt niet (Ctrl+Shift+B)
`Value cannot be null (Parameter 'connectionString')`	De naam in `GetConnectionString("...")` is ≠ de naam in appsettings.json
`Cannot open database "VoertuigDb"`	`Update-Database` vergeten
`Invalid object name 'Voertuigen'`	Migration niet toegepast. Doe `Add-Migration` + `Update-Database`
`Unable to resolve service for type 'VoertuigDbContext'`	`builder.Services.AddDbContext<...>` ontbreekt
`<car-info>` staat letterlijk in de HTML	`@addTagHelper *, MVCVoertuig` ontbreekt in `_ViewImports.cshtml` (projectnaam!)
`The view 'Components/HeaderMenu/Default' was not found`	Pad moet exact `Views/Shared/Components/HeaderMenu/Default.cshtml` zijn
`The partial view '_VoertuigCard' was not found`	Bestand staat niet in `Views/Shared/` of `Views/Voertuig/`, of de naam heeft een typfout
`InvalidOperationException: model item passed is List<...> but requires Voertuig`	`@model` bovenaan de view klopt niet: gebruik `IEnumerable<Voertuig>` voor een lijst
Validatiefout ⇒ niets gebeurt / geen meldingen	`<span asp-validation-for>` ontbreekt, of `return View(voertuig)` in de POST
Decimale prijs geeft "must be a number" of een fout bedrag	Belgische Windows verwacht een komma, de browservalidatie een punt. Typ dus **gehele getallen** (bv. `24950`)
Seed data wordt niet toegevoegd	`SeedData.EnsurePopulated(app);` ontbreekt, of staat **na** `app.Run()`
Model gewijzigd ⇒ fout "pending model changes"	Opnieuw `Add-Migration X` + `Update-Database`
Rode lijnen onder klassen	`using` ontbreekt: Ctrl + . op de rode lijn → kies de using

19. Checklist voor het indienen

- [ ] Packages (2) geïnstalleerd
- [ ] DbContext + DbSet
- [ ] Connection string (juiste naam + databanknaam)
- [ ] `AddDbContext` in Program.cs
- [ ] Model met alle annotaties uit de tabel
- [ ] Custom validatieklasse + attribuut boven de property
- [ ] Controller: Index + Create (GET + POST) + views
- [ ] `Add-Migration` + `Update-Database` → tabel bestaat
- [ ] Seed data + aanroep in Program.cs
- [ ] Layout volgens de opgave
- [ ] Partial view gebruikt in Index
- [ ] TagHelper + `@addTagHelper` + gebruikt in een view
- [ ] ViewComponent + Default.cshtml + aangeroepen in de layout
- [ ] Alles getest: starten, menu, create met foute en goede waarden